

## 实验四 寄存器组及存储器

2025 年秋季学期

此情可待成追忆，只是当时已惘然。

— 《锦瑟》，李商隐

寄存器组（Register File）与存储器（Memory）是数字系统中的记忆器件，用来存放程序和数据。从程序员的角度来看，CPU 的状态由其寄存器及存储器中的信息唯一确定。其中寄存器包括程序计数器 PC、通用寄存器，存储器指主存。我们可以将计算机看成一个巨大的有限状态自动机，当这些存储部件中的信息确定后，计算机的状态也确定了。在没有外部输入时，计算机后续的运行状态也是唯一确定的。

本实验的目的是了解 FPGA 的触发器及片上存储器的特性，分析存储器的工作时序和结构，并学习如何设计寄存器组和主存。

### 4.1 寄存器与寄存器组

FPGA 上有大量的触发器资源来实现数据的存储。D 触发器可以用于存储比特信号，给 D 触发器加上置数功能就变成了一位寄存器，如图 4-1 所示。由图中可以看出，如果 load 信号为 1，则输入信号 in 被送入或门中，或门的另一个输入端为 0，此时  $D=in$ ，所以在下一个时钟里  $q=in$ 。当 load 值为 0 时，q 值被反馈到或门中，或门的另一个输入值为 0，此时  $D=q$ ，因此在下一个时钟周期里 q 值保持先前的值不变。

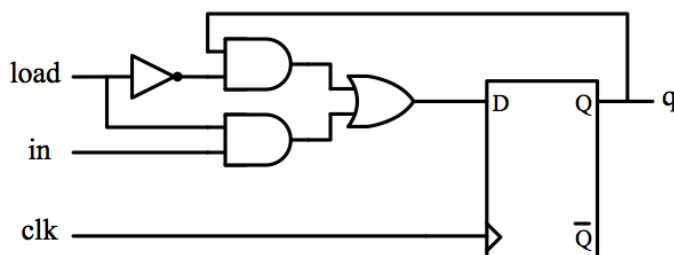


图 4-1: 1 位寄存器

用 Verilog 语言设计寄存器也很简单，如表 4-1 所示。

表 4-1: 1 位寄存器代码

```

1 module register1(load,clk,clr,inp,q);
2     input  load,clr,clk,inp;
3     output reg q;
4
5     always @(posedge clk)
6         if (clr==1)
7             q <= 0;
8         else if (load == 1)
9             q <= inp;
10 endmodule

```

表 4-1 的程序的仿真图如图 4-2 所示。

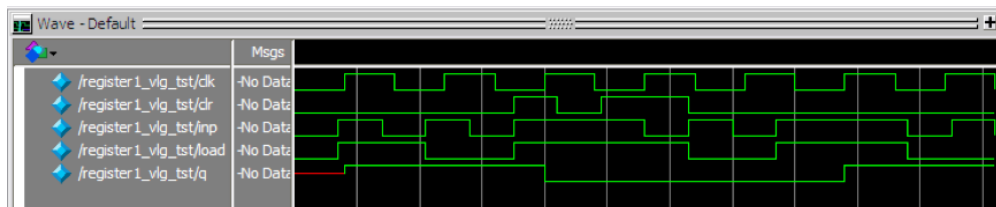


图 4-2: 1 位寄存器仿真结果

本例实现的是一个带有清 0 端和输入端的 1 位寄存器，还有的寄存器带有置位（置 1）端的，图 4-3 是同时带有清 0 端、输入端和置位端的寄存器的逻辑示意图，读者可自行设计此寄存器。

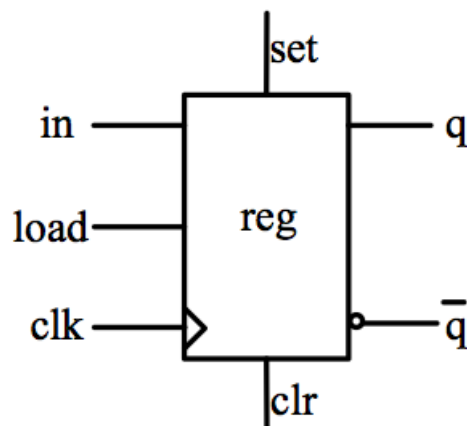


图 4-3: 1 位寄存器框图

将 2 个或者 2 个以上的 1 位寄存器组合在一起，这些寄存器共用一个时钟信号，这就构成了多位寄存器，寄存器常被用在计算机中存储数据，如指令寄

存器、数据寄存器等。表 4-2是利用 Verilog 语言设计寄存器的例子。

表 4-2: 4 位寄存器代码

```
1 module register4(load,clk,clr,d,q);
2     input  load,clr,clk;
3     input  [3:0] d;
4     output reg [3:0] q;
5
6     always @(posedge clk)
7         if (clr==1)
8             q <= 0;
9         else if (load == 1)
10            q <= d;
11 endmodule
```

表 4-2的程序的仿真图如图 4-4所示。

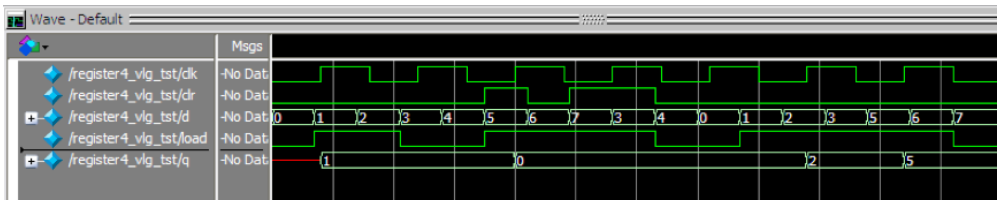


图 4-4: 4 位寄存器仿真结果

## 4.2 存储器结构

存储器是一组存储单元，用于在计算机中存储二进制的数 据，如图 4-5所示。存储器的端口包括：输入端、输出端和控制端口。输入端口包括：读/写地址端口、数据输入端口等；输出端口一般指的是数据输出端口；控制端口包括时钟端和读/写控制端口。存储器的工作过程如下：

**写数据：**在时钟（clk）有效沿（上升或下降沿），如果写使能（Wr\_en，也可以没有使能端）有效，则读取输入总线（Data\_in）上的数据，将其存储到输入地址线（In\_addr）所指的存储单元中。

**读数据：**存储器的输出可以受时钟和使能端的控制，也可以不受时钟和使能端的控制。如果输出受时钟的控制，则在时钟有效沿，将输出地址所指示的单元中的数据，输出到输出总线上（Data\_out）；如果不受时钟的控制，则只要输出地址有效，就立即将此地址所指的单元中的数据送到输出总线上。

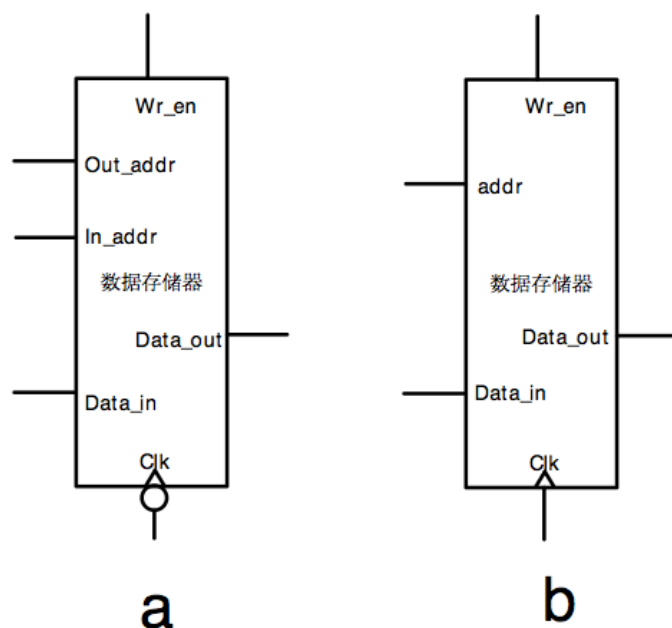


图 4-5: 存储器结构

对于存储器，其读写时序非常重要，也是实践中容易出错的地方。读取数据时在哪个时间点数据有效，写入数据过多久可以读取这些都要在设计时反复检查和验证。

FPGA 存储器的工作模式有很多，如：真双口 RAM、简单双口 RAM、单口 RAM、ROM 或者 FIFO 缓存等。常见的模式请参照下表。

在 Verilog HDL 中，可以用多维数组定义存储器。例如，假设需要一个 32 字节的 8 位存储器块，即此存储器共有 32 个存储单元，每个存储单元可以存储一个 8 位的二进制数。这样的存储器可以定义为 32×8 的数组，在 Verilog 语言中可以作如下变量声明：

```
1 Reg [7:0] memory_array [31:0];
```

存储单元为 memory\_array [0]~memory\_array [31]，每个存储单元都是 8 位的存储空间。在读取时，可以用 memory\_array [13][3:0] 直接读取第 13 号单元的低 4 位。

#### 寄存器与存储器的异同

虽然寄存器和存储器都是用来存储状态信息的，但是它们在用途和实现上较大的区别：

表 4-3: 存储器的工作模式

| 存储器模式          | 说明                                          |
|----------------|---------------------------------------------|
| 单口存储器          | 某一时刻，只读或者只写                                 |
| 简单双口存储器模式      | 简单双口模式支持同时读写（一读一写）                          |
| 混合宽度的简单双口存储器模式 | 读写使用不同的数据宽度的简单双口模式                          |
| 真双口存储器模式       | 真双口模式支持任何组合的双口操作：两个读口、两个写口和两个不同时钟频率下的一读口一写口 |
| 混合宽度的真双口存储器模式  | 读写使用不同的数据宽度的真双口模式                           |
| ROM            | 工作于 ROM 模式，ROM 中的内容已经初始化                    |
| FIFO 缓冲器       | 可以实现单时钟或双时钟的 FIFO                           |

- 寄存器一般要求存取速度快、并行访问要求高，所以通常寄存器的容量较小。在 CPU 中，PC 及通用寄存器会经常被访问，因此存取的时延要求在一个时钟周期内。对于单周期 CPU，每个时钟周期往往要求同时读取 2 个通用寄存器并完成 1 个寄存器的写回。在要求较高的时候，有可能寄存器组输出的结果需要异步输出，即不在时钟沿上读取，输出随着输入地址实时改变。在这样高的要求下，寄存器组的大小不可能太大，否则会消耗非常多的资源。
- 主存一般容量较大，但是读写时间较长，并且读写过程有严格的时序要求。
- 在 Verilog 中，虽然寄存器组和存储器的描述都是二维数组的方式。但是，编译和综合过程中会根据代码访问的要求来选择具体的实现方式。例如，当代码中没有严格在时钟信号沿上进行读写时，系统会认为该存储单元的读写要求较高，直接采用 FPGA 逻辑单元实现。这种实现方式消耗的资源巨大，一般只能支持数 K 量级的存储单元。如果要求大量的此类存储功能，系统可能会花很长时间进行编译综合，甚至无法实现。如果一个存储单元的访问严格按照时序要求，仅在时钟沿上进行每次单个单元的读写时，系统可以用大容量的 BRAM 实现存储，一般可以支持到数百 K 字节的容量。因此，在实验中对存储器的读写应特别关注，避免用高级语言的二维数组的思路来看待存储器，否则会造成很多意想不到的后果。

### 4.3 存储器的实现

Xilinx Artix-7 100T FPGA 中存储器有两种实现方式，一种是分布式 RAM (Distributed RAM)，利用 LUT 和周围的 DFF 来实现，Artix-7 100T FPGA 有 15,850 个逻辑片 Slice，每个 Slice 包含 4 个 6 输入的 LUT 和 8 个触发器，最大分布式 RAM 容量为 1,188kb。另一种是专用集成块 RAM (Block RAM)，称为 BRAM。Artix-7 100T FPGA 中有 135 个块 RAM，每个块 RAM 容量 36kb，共计 4,860Kb 的高速 BRAM。在容量方面，BRAM 可用的容量比分布式 RAM 大的多。在时序方面，BRAM 读和写操作上均需要 1 个时钟周期，而分布式 RAM 写操作需要 1 个时钟，读操作可以是异步读取。当存储器需要使用的较少存储空间且时序要求不高时，则使用分布式 RAM，以便节省 LUT 逻辑资源。BRAM 适用于需求较大的存储空间，在实例化一个 BRAM 后，即使只是占用到该 RAM 的一小部分，而综合后，可能会消耗一整块 RAM 的资源 (Artix-7 100T 中每个块 RAM 可以配置为两个独立的 18KbRAM)，BRAM 在 FPGA 中是按列分布，可能造成用户逻辑和 BRAM 之间的布线较长和延时增加。

表 4-4: 存储器实现代码

```

1 module myram #(
2     parameter RAM_WIDTH = 32,
3     parameter RAM_ADDR_WIDTH = 10
4 )(
5     input clk,
6     input we,
7     input [RAM_WIDTH-1:0] din,
8     input [RAM_ADDR_WIDTH-1:0] inaddr,
9     input [RAM_ADDR_WIDTH-1:0] outaddr,
10    output [RAM_WIDTH-1:0] dout
11 );
12
13    reg [RAM_WIDTH-1:0] ram [(2**RAM_ADDR_WIDTH)-1:0];
14
15    always @(posedge clk)
16        if (we)
17            ram[inaddr] <= din;
18
19    assign dout = ram[outaddr];
20
21 endmodule

```

### 4.3.1 存储器实例分析

表4-5是一个存储器实例，实例中为此存储器设置了三个输出端口，请分析存储器结构和工作过程，查看此存储器的 RTL 图，检查存储器的输入输出和存储体的结构，并分析其三个输出端的结构的不同。为此实例设计一个测试代码，研究此三个端口输出数据时在时序上的差别，结合 RTL 图，给出其工作时序的解释。注意，如果在开发板上直接观察 RAM 的时序，建议将 RAM 的读写时钟改为按钮手动实现或者 1 秒时钟，避免时钟过快无法观察。

表 4-5: 存储器实例代码

```

1 module v_rams_8 (clk, we, inaddr, outaddr, din, dout0,dout1,dout2);
2 input clk;
3 input we;
4 input [2:0] inaddr;
5 input [2:0] outaddr;
6 input [7:0] din;
7 output reg [7:0] dout0,dout1;
8 output [7:0] dout2;
9
10 reg [7:0] ram [7:0];
11
12 initial
13 begin
14 ram[7] = 8'hf0; ram[6] = 8'h23; ram[5] = 8'h20; ram[4] = 8'h50;
15 ram[3] = 8'h03; ram[2] = 8'h21; ram[1] = 8'h82; ram[0] = 8'h0D;
16 end
17
18 always @(posedge clk)
19 begin
20     if (we)
21         ram[inaddr] <= din;
22     else
23         dout0 <= ram[outaddr];
24 end
25 always @(negedge clk)
26 begin
27     if (!we)
28         dout1 <= ram[outaddr];
29 end
30 assign dout2 = ram[outaddr];
31 endmodule

```

其中 initial 语句块完成了在启动时对 RAM 的初始化。

适当选择输入输出端口宽度，将此实例进行引脚约束，利用开关或按钮作为时钟端，在开发板上再次验证其不同输入/输出方式的工作时序。

### 4.3.2 存储器初始化

当需要初始化的 RAM 数据量较大的时候，可以使用文件来在系统启动时直接装入 RAM 数据。Verilog 提供了以下语句来将文件中的数据导入到 RAM 中：

```
1 initial
2 begin
3     $readmemh("D:/digital_logic/mem1.txt", ram, 0, 7);
4 end
```

以上内容可以替代前例中的 RAM 初始化部分，将 mem1.txt 中的数据导入到 ram 变量的第 0 单元至第 7 单元。请注意，这里 mem1.txt 可以存在任何不包含中文字符的目录下，但是在初始化语句中一定要给出此文件的绝对路径，否则仿真时将看不到初始化数据。

mem1.txt 的内容和格式如下：

```
1 @0 0d
2 @1 82
3 @2 21
4 @3 03
5 @4 20
6 @5 ff
7 @6 50
8 @7 04
```

其中 @ 符号后为 ram 地址，随后是 16 进制的 ram 数据。在 verilog 中，\$readmemh 方法读取 16 进制数据，\$readmemb 方法读取 2 进制数据。

初始化存储器时可以选择存储器的部分单元进行初始化，其他单元不初始化。如，假设存储器 ram 有 8 个存储单元，下面的初始化表示只对存储器的 0~5 号单元进行初始化，这也是可以的。

```
1 initial
2 begin
3     $readmemh("D:/digital_logic/mem1.txt", ram, 0, 5);
4 end
```

假设存储器 ram 有 8 个存储单元，下面的初始化试图对存储器的 0~8 号单元，共 9 个单元进行初始化，这是**不可以**的。

```

1 initial
2 begin
3     $readmemh("D:/digital_logic/mem1.txt", ram, 0, 8);
4 end

```

对存储器进行初始化还有其他方式，我们会在以后的实验中继续介绍。

## 4.4 使用 IP 核生成存储器

Vivado 提供了很多非常实用的 IP 核，IP 核是预先实现和测试过的模块，具有较高的可靠性。利用这些 IP 核可以很方便的实现复杂的设计。下面我们设计一个存储器为例来介绍如何使用 Vivado IP 核。

### 4.4.1 通过 IP 生成 RAM

在 Vivado 界面的左边 Flow Navigator 下的 Project Manager 里点击 IP Catalog，界面右边会显示 IP 目录，搜索 RAM，可以找到 Block Memory Generator 如图 4-6 所示

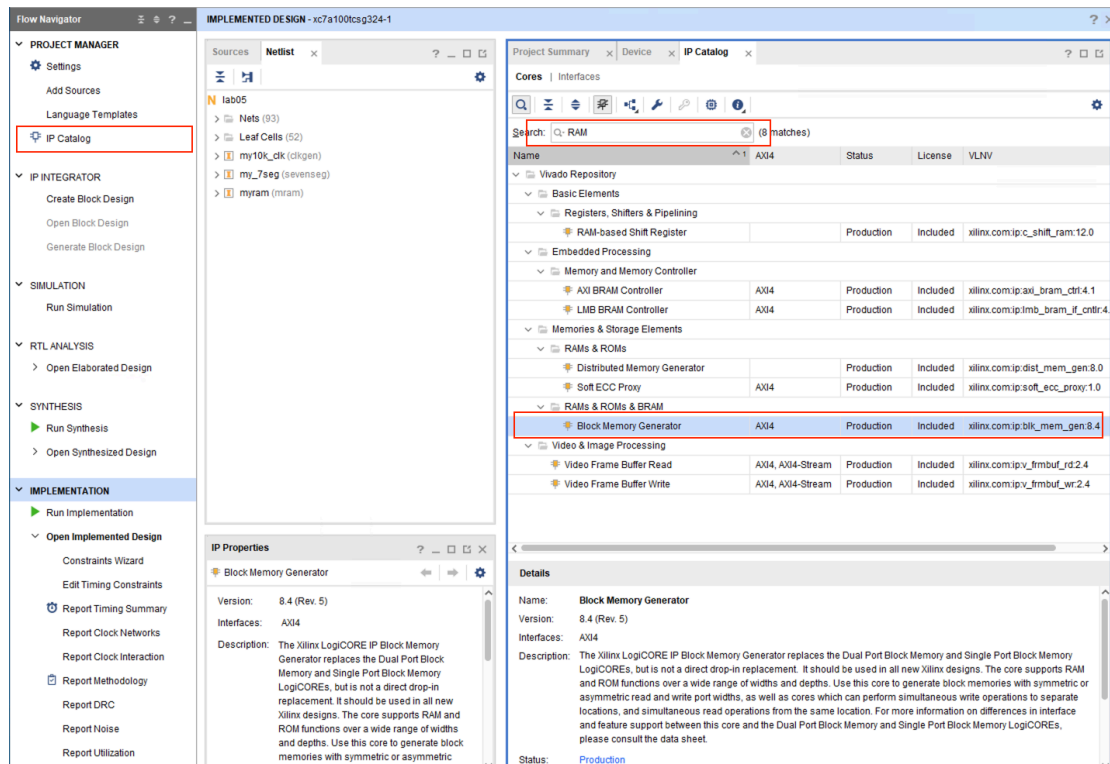


图 4-6: IP 目录

点击 **Block Memory Generator** 可以进入如图 4-7 界面对存储器进行详细配置。

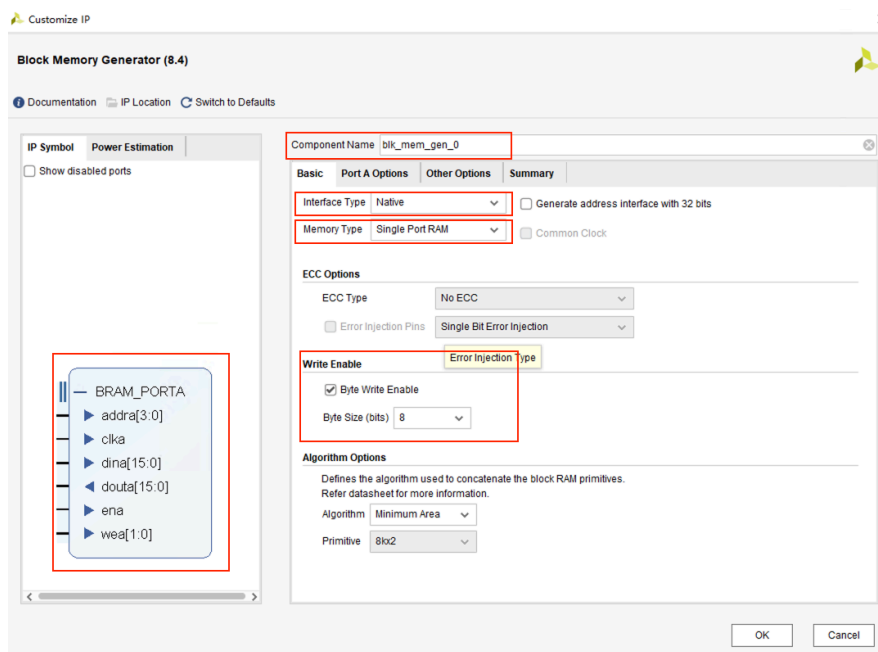


图 4-7: 存储器基础配置

Vivado 的存储模块 IP 配置有较多选项，请务必清晰理解每个选项的含义，避免配置出不合理的结果。下面我们按照配置一个 16 个存储单元，每个存储单元 16bit（2Byte）的 RAM 为例进行说明。在图 4-7 中的基础配置中包含以下的重要选择项。

- **Component Name:** 决定了本 IP 模块的名称，后续实例化该模块时需要按此模块命名来实例化，这里我们使用了缺省名称 `blk_mem_gen_0`。
- **Interface Type:** 定义了存储模块的接口，即实例化之后该模块有哪些输入输出参数。Vivado 的存储模块支持 **Native** 的基本存储模块，即类似我们之前的 RAM 实现，以读写地址、写使能、写入数据、输出读数据及时钟这些接口调用存储模块，也可以使用标准的 **AXI** 总线接口调用存储。我们这里使用 **Native** 接口，后续在计算机系统中可能会采用总线调用方式。接口选择后可以点开左边的模块接口示例，观察该模块有哪些接口。当配置不同时，模块的接口及接口的位宽都有可能改变。
- **Memory Type:** 支持单口 RAM，真双口 RAM，单口 ROM 及双口 ROM 的存储模式。不同存储模式提供不同数量的读写端口。
- **Write Enable:** 写使能，可以采用一位写使能对整个单元进行写入，也可以

根据需要写入部分数据。例如，我们可以勾选 **Byte Write Enable** 选项，让数据每 8 个 bit 拥有单个写使能。这样，我们的 16bit RAM 就会有两位的写使能 `wea[1:0]`。当 `wea[0]` 为 1 时对低 8bit 写入，`wea[1]` 为 1 时对高 8 比特写入，如果要对 16bit 写入则可以设置 `wea[1:0]=2'b11`。这种 **Byte Write** 将来在实现 RISC-V 存储器中对 32 位的 **Word** 进行单字节操作的时候非常有用。

在图 4-8 中 Port A Option 里的配置主要是：

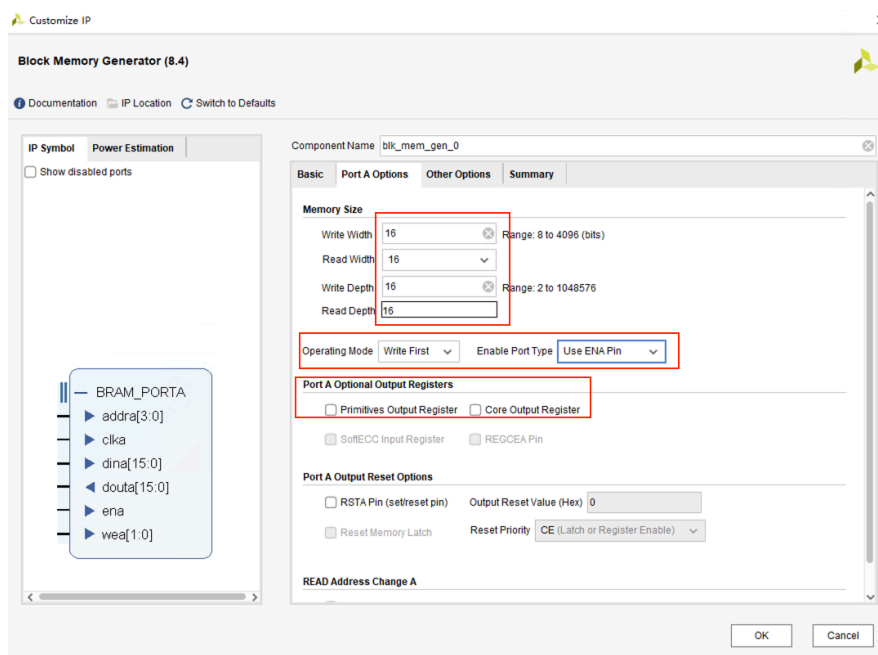


图 4-8: Port A 配置

- 存储器大小：主要通过 **Read/Write Width** 来配置每个单元的 bit 数，我们这里统一配置为 16 个 bit。通过 **Read/Write Depth** 来配置单元数量，这里选择 16 个存储单元，即读写地址是 4 位的。单元数量可以是 2 的幂次，也可以自行选择以减小存储的小孩。
- 操作模式及使能方式：用于配置是写优先还是读优先，以及片选使能端的方式，可以使用缺省设置。
- 输出寄存器：配置是否需要在输出端进行寄存，如果选择了输出寄存，可以观察最后的 **Summary** 会发现读取所需的周期数会由原来的 1 个周期变为 2 个周期，这主要是输出寄存器需要一个周期的时延。所以，我们这里选择不要输出寄存，即一个周期就可以得到数据，这在未来的 CPU 读取时序设计中非常重要。
- 其他配置：可以使用缺省值。

在图4-9中的其他属性配置里最重要的是初始化文件的选择。在 Vivado 中，可以使用 Xilinx 自己专用的 COE 文件格式来对存储进行初始化，COE 文件也是用文本方式来规定存储的内容。例如，我们可以用如下方式来设置 COE 文件：

```
;.COE file with hex coefficients
memory_initialization_radix=16;
memory_initialization_vector=
0000,0001,0002,0003,
0004,0005,0006,0007,
0008,0009,000A,000B,
000C,000D,000E,000F;
```

其中以;号开头的行为注释，memory\_initialization\_radix 规定了存储初始化文件使用 16 进制方式表示。memory\_initialization\_vector 规定了所有单元的内容，存储单元以逗号分开，最后以分号结束。

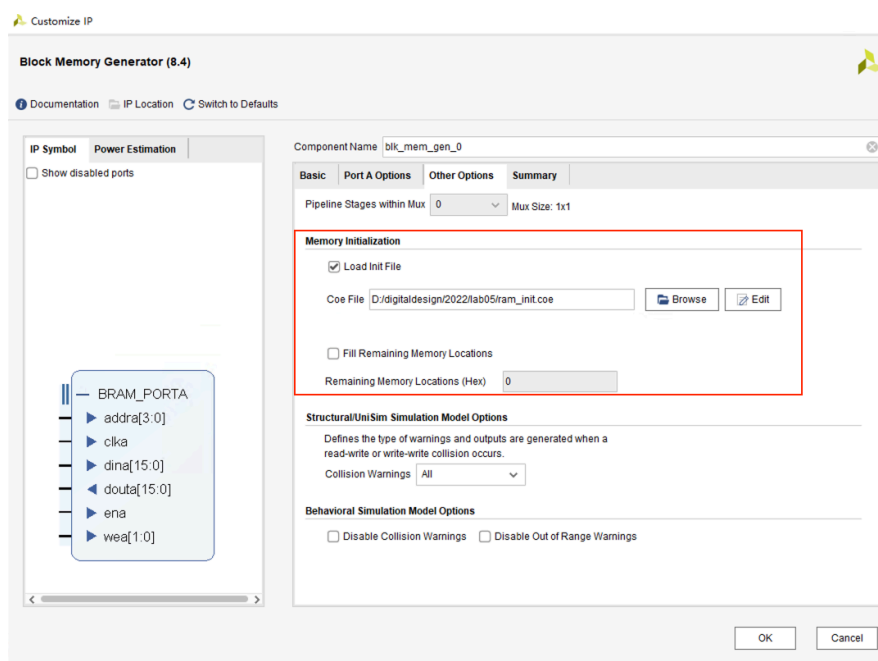


图 4-9: 其他配置

在图4-10中的 IP 核配置结果中显示了该存储模块使用了多少 BRAM 资源，以及读取需要多少时钟周期。选择 **OK** 后，在下一步的对话框中点击 **Generate**。等生成完毕，该 IP 核就可以使用了，我们可以在 Sources 里的列表中观察到对应的模块，如图4-11所示。在完成配置之后，如果需要修改 IP

配置也可以双击 Source 中对应的 IP 核进行修改。

在调用 IP 核时，可以按一般的模块实例化方式进行实例化，如：

```
blk_mem_gen_0 myram(.addra(SW[15:12]),  
                    .clka(clk),  
                    .dina({8'h0,SW[7:0]}),  
                    .douta(LED[15:0]),  
                    .ena(1'b1),  
                    .wea(SW[11:10]));
```

将该 IP 核实例化，并连接到开发板的各个输入输出上去。

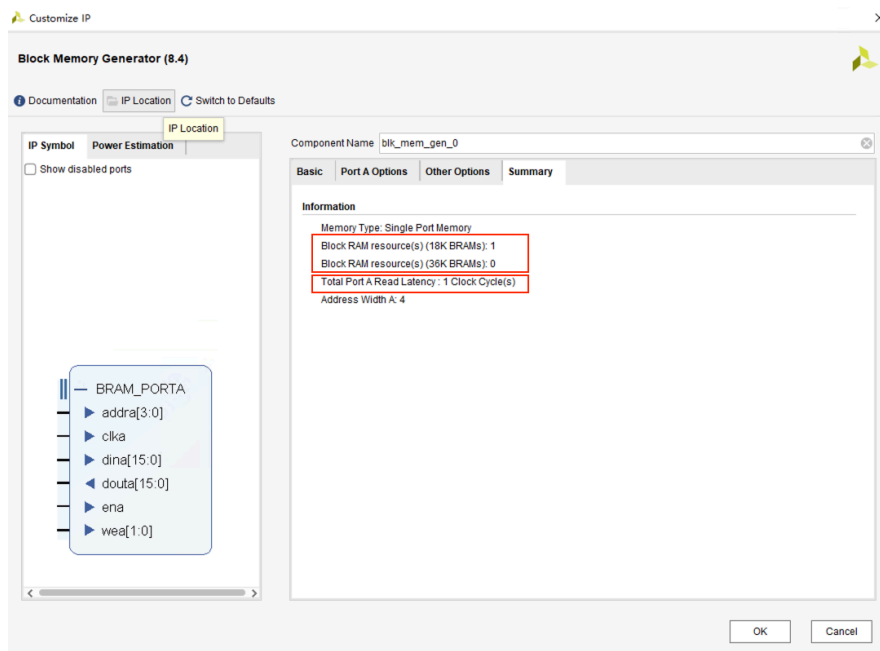


图 4-10: IP 核配置结果

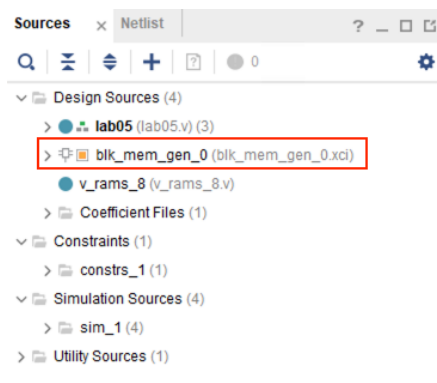


图 4-11: Source 中的 IP 核

## 4.5 实验内容

### 4.5.1 上板验收

请在工程中完成如下的寄存器堆和 RAM。寄存器堆和 RAM 的大小均为  $16 \times 8$ ，即都有 16 个存储单元，每个存储单元都是 8 位的，均可以进行读写。

✎ **寄存器堆**：读取时不需要时钟控制，即读地址有效后，直接输出数据。写入时通过时钟上升沿进行控制。

此时可用以下方式输出：

```
1 assign out = ram[addr];
```

采用下面的方式进行初始化。

```
1 initial
2 begin
3     $readmemh("D:/digital_logic/mem1.txt", ram, 0, 15);
4 end
```

初始化数值为

```
1 @0 00
2 @1 01
3 @2 02
4 @3 03
5 @4 04
6 @5 05
7 @6 06
8 @7 07
9 @8 08
10 @9 09
11 @a 0a
12 @b 0b
13 @c 0c
14 @d 0d
15 @e 0e
16 @f 0f
```

✎ **RAM**: 利用 IP 核设计一个单口存储器, 利用 .coe 文件进行初始化, 十六个单元的初始化值分别为: 0xf0, 0xf1, 0xf2, 0xf3, 0xf4, 0xf5, 0xf6, 0xf7, 0xf8, 0xf9, 0xfa, 0xfb, 0xfc, 0xfd, 0xfe, 0xff。

此两个物理上完全不同的存储器共用时钟、读写地址。适当选择时钟信号和写使能信号, 以能够分别对此两个存储器进行读写。请将两个存储器读出的结果分别用 2 个七段数码管显示。请合理使用 FPGA 开发板的输入/输出资源, 完成此寄存器堆和 RAM 的设计。由于开发板上输入数量不够, 写入时可以只写入 4 位数据。

✎ 请使用开发板上的按钮来做为存储器的时钟信号。观察两个不同的实现方式下各需要几个时钟周期来完成读取或写入操作?

✎ 在实现之后 Open Implementation Design, 观察 Schematic 中你生成的两个存储器, 点击存储器模块直至最内层, 观察综合后这两个存储器分别使用了什么方式来实现, 为什么?

✎ 注意观察综合后输出的资源消耗, 在 Implement 之后选择 Report Utilization, 图 4-12 中显示我们用 IP 核生成的 RAM 使用了多少 Block RAM? 观察寄存器堆使用了哪些资源。

| Name                  | Slice LUTs<br>(63400) | Slice Registers<br>(126800) | Slice<br>(15850) | LUT as Logic<br>(63400) | Block RAM<br>Tile (135) | Bonded IOB<br>(210) | BUFGCTRL<br>(32) |
|-----------------------|-----------------------|-----------------------------|------------------|-------------------------|-------------------------|---------------------|------------------|
| lab05                 | 27                    | 36                          | 21               | 27                      | 0.5                     | 47                  | 1                |
| my10k_clk (clkgen)    | 12                    | 33                          | 15               | 12                      | 0                       | 0                   | 0                |
| my_7seg (sevenseg)    | 14                    | 3                           | 7                | 14                      | 0                       | 0                   | 0                |
| myram (blk_mem_gen_0) | 1                     | 0                           | 1                | 1                       | 0.5                     | 0                   | 0                |

图 4-12: 系统资源消耗

## 4.5.2 在线测试

必做 寄存器堆实现

选做 前导零个数判断